TRILHA DE CONHECIMENTO / JAVA BÁSICO

NovaBank

Sistema bancário simples com Java no console

Fundamentos de Java e prática com Docker
Enunciado e guia de entrega para alunos

PRATICAR

Variáveis, condicionais, laços e métodos

ORGANIZAR

Classes, objetos e listas

CONSTRUIR

Cadastros, operações e histórico

Nível básico | Projeto educacional | Outubro de 2026

NOVABANK / PROJETO PRÁTICO / 01

Guia do projeto

Crie um sistema bancário simples em Java, executado no console, para praticar os fundamentos da linguagem.

A instituição fictícia **NovaBank** precisa de um programa para cadastrar clientes, criar contas e realizar operações simples. O aluno deverá construir um menu que permita usar essas funcionalidades durante a execução.

O que será praticado

Conceito	Aplicação no projeto
Variáveis e tipos de dados	Guardar nome, CPF, número da conta e saldo.
Condicionais	Verificar valores, saldo disponível e opção escolhida.
Laços de repetição	Manter o menu ativo e percorrer os cadastros.
Métodos	Organizar cadastro, busca, depósito, saque e transferência.
Classes e objetos	Representar clientes e contas.
Encapsulamento	Proteger o saldo e controlar sua alteração por métodos.
Listas	Armazenar clientes, contas e histórico em memória.
Docker e Docker Compose	Criar a imagem e executar o programa em container.

Escopo desta versão

Use Java e a biblioteca padrão. A interação acontece pelo terminal, com entrada de dados e mensagens em português. Os dados podem permanecer apenas na memória: ao fechar o programa, é permitido perder os cadastros e os saldos.

Pré-requisitos: fundamentos de Java, introdução a classes e objetos e conceitos de Docker já praticados na trilha. Use somente dados fictícios.

NOVABANK / PROJETO PRÁTICO / 02

Clientes e contas

Comece pelos cadastros e consultas. As operações financeiras dependerão dessas informações.

Clientes: funcionalidades obrigatórias

• Cadastrar um cliente com nome e CPF. E-mail e telefone são opcionais.

• Listar os clientes cadastrados e buscar um cliente pelo CPF.

• Não permitir nome vazio, CPF vazio ou CPF já cadastrado.

CONSOLE | EXEMPLO DE CADASTRO

```
Nome: Maria Silva
CPF: 52998224725

Cliente cadastrado com sucesso!
```

Para simplificar, armazene o CPF como String e aceite somente 11 dígitos, sem pontos ou traço. Nesta etapa, não é necessário calcular os dígitos verificadores. Use dados fictícios nos exemplos.

Contas: funcionalidades obrigatórias

• Criar uma conta associada a um cliente já cadastrado.

• Gerar um número de conta único. Uma sequência simples, como 1001, 1002 e 1003, é suficiente.

• Iniciar toda conta com saldo zero. Um cliente pode ter mais de uma conta.

• Listar as contas e consultar uma conta pelo número, mostrando titular e saldo.

CONSOLE | CONSULTA DE CONTA

```
Número da conta: 1001
Titular: Maria Silva
Saldo: R$ 0,00
```

NOVABANK / PROJETO PRÁTICO / 03

Operações bancárias

Implemente as operações após concluir os cadastros. Cada operação deve validar seus dados antes de alterar o saldo.

Operação obrigatória	Regra mínima
Depósito	Conta existente e valor maior que zero. Somar o valor ao saldo.
Saque	Conta existente, valor maior que zero e saldo suficiente. Subtrair o valor do saldo.
Transferência	Duas contas existentes e diferentes, valor maior que zero e saldo suficiente na origem. Debitar a origem e creditar o destino.
Histórico simples	Consultar as operações de uma conta na ordem em que foram realizadas.

Uma forma simples de representar dinheiro

Para este exercício, use **long em centavos** para os valores e o saldo. Assim, 15000 representa R$ 150,00 e 1050 representa R$ 10,50. Solicite valores inteiros em centavos no menu e mostre o saldo formatado em reais.

CONSOLE | ENTRADA E SAÍDA

```
Valor do depósito em centavos: 15000
Depósito realizado com sucesso!
Saldo atual: R$ 150,00
```

Não aceite zero, valores negativos ou entrada acima da capacidade do tipo escolhido. Ao somar um depósito ou crédito, verifique se o resultado cabe em long. Para o nível básico, registre data/hora apenas se já tiver estudado o assunto.

Histórico de movimentações

Uma List<String> por conta é suficiente. Registre tipo, valor e saldo após cada operação. A transferência deve gerar um registro na origem e outro no destino. Operações rejeitadas não alteram saldo nem histórico.

NOVABANK / PROJETO PRÁTICO / 04

Classes e métodos

Use poucas classes, com responsabilidades claras. A estrutura abaixo é uma sugestão para apoiar quem está começando.

Classe sugerida	Dados e responsabilidades
Cliente	Nome e CPF; representa a pessoa que possui a conta.
Conta	Número, cliente, saldo em centavos e histórico. Métodos para consultar saldo, depositar e sacar.
Aplicacao	Método main, menu, leitura de dados, listas de cadastros e busca por CPF/número de conta.

Organize o código em etapas pequenas

- Mantenha os atributos da conta privados. Evite alterar saldo diretamente no menu.
- Use métodos com nomes claros: cadastrarCliente, criarConta, buscarConta, depositar e sacar.
- Organize a transferência em um método que valide as duas contas e depois execute os ajustes.
- Busque itens nas listas com um for e compare CPF com equals.

JAVA | EXEMPLO PARCIAL DE MÉTODO

```
public boolean sacar(long valorEmCentavos) {
    if (valorEmCentavos <= 0) {
        return false;
    }
    if (valorEmCentavos > saldoEmCentavos) {
        return false;
    }

    saldoEmCentavos -= valorEmCentavos;
    return true;
}
```

O exemplo pertence à classe Conta e pressupõe o atributo saldoEmCentavos. O retorno indica se o saque foi aceito. Ainda será necessário registrar o histórico e apresentar a mensagem adequada ao usuário. O trecho é uma orientação, não a solução completa.

NOVABANK / PROJETO PRÁTICO / 05

Menu e interação

O usuário deve conseguir realizar todas as operações pelo console e encerrar o programa quando desejar.

CONSOLE | MENU SUGERIDO

```
========== NOVABANK ==========
1 - Cadastrar cliente
2 - Listar clientes
3 - Buscar cliente por CPF
4 - Criar conta
5 - Listar contas
6 - Consultar saldo
7 - Realizar depósito
8 - Realizar saque
9 - Realizar transferência
10 - Consultar histórico
0 - Sair
Escolha uma opção:
```

Comportamento esperado

• Use um laço para repetir o menu até a opção 0.

• Use switch ou if/else para executar a opção escolhida.

• Após cada operação, mostre o resultado e permita continuar usando o programa.

• Se a opção não existir, informe o problema e apresente o menu novamente.

• Se uma conta ou um cliente não for encontrado, mostre uma mensagem clara.

• Não encerre inesperadamente o programa quando o usuário digitar um texto no lugar de um número.

Entrada de dados

Scanner é suficiente para ler as entradas. Uma estratégia é usar nextLine para todas as leituras e converter o conteúdo quando for numérico. Se utilizar nextInt ou nextLong, atenção à quebra de linha que pode permanecer antes da próxima leitura com nextLine.

NOVABANK / PROJETO PRÁTICO / 06

Roteiro de desenvolvimento

Construa uma funcionalidade por vez e confira seu comportamento antes de seguir para a próxima.

Etapa	O que implementar	Como conferir
1. Menu	Classe com main e menu repetido em um laço.	Escolher opções e sair com 0.
2. Clientes	Classe Cliente, cadastro, listagem e busca.	Cadastrar dois clientes e localizar cada um.
3. Contas	Classe Conta, criação e consulta.	Abrir contas e confirmar saldo zero.
4. Depósito e saque	Métodos, validações e atualização de saldo.	Testar valores válidos e recusados.
5. Transferência	Busca das contas e validação antes dos ajustes.	Conferir os dois saldos após a operação.
6. Histórico	Registro das operações de cada conta.	Consultar registros e saldos registrados.
7. Container	Dockerfile, Compose e imagem da aplicação.	Usar o menu dentro do container.
8. Revisão	Entradas inválidas, organização e README.	Repetir os cenários da página 8 no container.

Diferenciais opcionais para esta etapa

- Adicionar e-mail e telefone; permitir atualização dos dados do cliente.
- Bloquear e desbloquear uma conta, impedindo movimentações enquanto bloqueada.
- Criar uma classe Movimentacao para substituir os registros de texto.
- Salvar ou exportar o histórico em um arquivo TXT.
- Usar enums para identificar o tipo de movimentação, se o tema já tiver sido estudado.

Não há prazo fixado neste material. O instrutor poderá definir a data de entrega e orientar o uso dos recursos já apresentados em aula.

NOVABANK / PROJETO PRÁTICO / 07

Validação da solução

Faça os testes manualmente pelo console. O aluno deve conseguir explicar o resultado de cada cenário.

Cenário	Resultado esperado
Cadastro com nome ou CPF vazio	Rejeitar e apresentar mensagem.
CPF já cadastrado	Rejeitar o novo cadastro.
Criar conta para cliente inexistente	Rejeitar a abertura de conta.
Consultar conta recém-criada	Mostrar saldo zero.
Depositar 10000 centavos	Mostrar saldo de R$ 100,00.
Sacar 3000 centavos dessa conta	Mostrar saldo de R$ 70,00.
Sacar 8000 centavos com saldo 7000	Rejeitar; manter o saldo de R$ 70,00.
Depositar/sacar/transferir zero ou negativo	Rejeitar; preservar saldos e históricos.
Transferir 2000: origem 7000, destino 0	Mostrar R$ 50,00 na origem e R$ 20,00 no destino.
Transferir para a mesma conta ou conta inexistente	Rejeitar; preservar os saldos.
Consultar histórico	Mostrar somente operações concluídas, na ordem correta.
Digitar opção inválida ou texto em campo numérico	Informar o problema e continuar o programa.
Digitar a opção 0	Encerrar o programa normalmente.

Registre as evidências

Inclua no README uma lista dos cenários verificados ou algumas capturas do console. Identifique entrada, resultado esperado e resultado observado. Testes automatizados são opcionais, caso já tenham sido apresentados na trilha.

Execute os cenários com docker run e com Docker Compose. Registre os comandos e os resultados observados.

NOVABANK / PROJETO PRÁTICO / 08

Container Docker

A criação da imagem e a execução da aplicação em container são requisitos obrigatórios desta atividade.

Conceito	Aplicação no exercício
Dockerfile	Receita para preparar o Java, copiar e compilar o código.
Imagem	Pacote criado a partir do Dockerfile para executar a aplicação.
Container	Instância da imagem em execução, com o menu interativo.

Dockerfile de referência

O exemplo considera os arquivos .java diretamente em src, a classe principal Aplicacao sem declaração de package e Java 21. Adapte os caminhos e o nome da classe se organizar o código de outra forma.

DOCKERFILE | NA RAIZ DO PROJETO

```
FROM eclipse-temurin:21-jdk
WORKDIR /app
COPY src/ ./src/
RUN javac -encoding UTF-8 -d out src/*.java
CMD ["java", "-cp", "out", "Aplicacao"]
```

Criar a imagem e iniciar o menu

TERMINAL | EXECUTAR NA RAIZ DO PROJETO

```
docker build -t novabank-java .
docker run --rm -it novabank-java
```

O build compila o código dentro da imagem. No run, -i mantém a entrada aberta, -t fornece o terminal e --rm remove o container quando a aplicação termina. A opção 0 deve encerrar o programa normalmente. [1]

Pré-requisitos: Docker instalado e em execução, com suporte a containers Linux, e terminal interativo para acessar o menu. A imagem já contém o JDK.

A imagem base é a Eclipse Temurin oficial. Se a trilha utilizar outra versão do Java, ajuste a tag do JDK e documente a versão adotada. O primeiro build precisa baixar a imagem base. [3]

NOVABANK / PROJETO PRÁTICO / 09

Docker Compose e execução

Entregue também um arquivo Compose para reproduzir a execução do programa com uma configuração simples.

DOCKER-COMPOSE.YML | NA RAIZ DO PROJETO

```
services:
  novabank:
    build: .
    image: novabank-java
    stdin_open: true
    tty: true
```

stdin_open e tty permitem a interação com o console. Para abrir o menu, utilize docker compose run no terminal. O comando cria uma execução do serviço e --rm remove esse container após a saída. [2]

TERMINAL | EXECUÇÃO COM COMPOSE

```
docker compose build
docker compose run --rm novabank
```

O que precisa funcionar

- Construir a imagem a partir de um clone ou cópia limpa do projeto.
- Visualizar o menu e informar dados pelo terminal dentro do container.
- Executar os cenários da página 8 e encerrar pela opção 0.
- Documentar que os cadastros e saldos em memória são perdidos ao encerrar.
- Reconstruir a imagem após alterar o código Java.

Este programa funciona pelo console; não é necessário publicar portas, configurar um banco ou criar volumes. Salvar arquivos e manter dados entre execuções continuam como evoluções opcionais.

Referências de apoio

[1] Docker: execução de containers

[2] Docker Compose: run e configuração do serviço

[3] Eclipse Temurin: imagem oficial

NOVABANK / PROJETO PRÁTICO / 10

Entrega e avaliação

Entregue o código-fonte, os arquivos de container e instruções para reproduzir a execução do projeto.

O que entregar

• Arquivos .java organizados, com nomes de classes e métodos claros.

• Dockerfile, docker-compose.yml e .dockerignore.

• README com versão do Java e comandos para build e execução interativa.

• Evidências dos cenários testados no container e do build da imagem.

• Publicação obrigatória no GitHub pessoal do aluno. A entrega deve ser feita pelo link do repositório.

ESTRUTURA | SUGESTÃO

```
banco-digital-java/
  src/  # Aplicacao.java, Cliente.java e Conta.java
  Dockerfile
  docker-compose.yml
  .dockerignore
  README.md
  .gitignore
```

No README, explique os pré-requisitos, os comandos Docker e Compose, o menu e os valores em centavos. No .dockerignore, exclua ao menos .git, out e arquivos .class. Maven e Gradle não são obrigatórios.

Critérios de avaliação sugeridos

Critério	Peso
Funcionalidades obrigatórias funcionando	30%
Regras de negócio e validações	20%
Uso de classes, métodos e encapsulamento	20%
Organização do código e clareza do menu	10%
Container: Dockerfile, Compose e execução	15%
README e evidências de testes	5%